

Chat System — Service Discovery Deep Dive

1. Why Deep Dive is Important

In system design interviews, after presenting the high-level architecture, interviewers usually expect deeper discussion on important components.

For chat systems, the key deep dive topics are:

- Service discovery
- Messaging flow
- Online/offline indicators

2. What is Service Discovery?

Service discovery is responsible for:

None

Selecting the best chat server for a client

3. Main Purpose of Service Discovery

When millions of users connect simultaneously:

- Clients must be distributed intelligently
- Servers must avoid overload
- Connections should remain stable

Service discovery solves these problems.

4. Selection Criteria

The best chat server is selected using factors such as:

Geographical Location

Users connect to nearby servers for:

- Lower latency
- Faster messaging
- Better user experience

Server Capacity

The system checks:

- CPU usage
- Memory usage
- Active WebSocket connections

before assigning a server.

Network Conditions

A server with better network health may be preferred.

5. Popular Solution — Apache Zookeeper

A commonly used open-source solution is:

None

Apache Zookeeper

6. Role of Zookeeper

Zookeeper acts as a:

None

Centralized Service Registry

It maintains information about all available chat servers.

7. What Zookeeper Stores

It keeps track of:

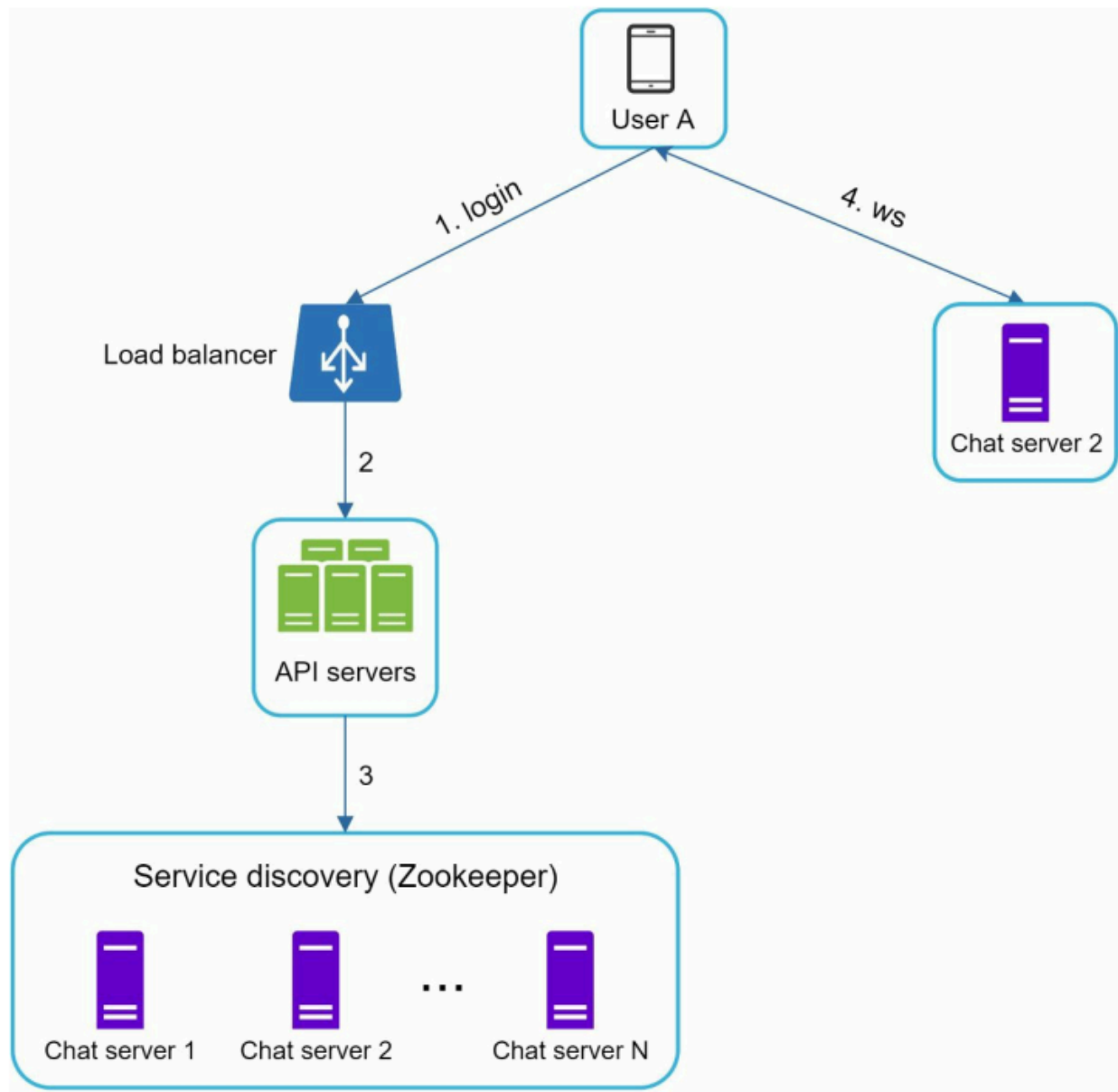
- Active chat servers
- Server health
- Connection count
- Server metadata
- Availability status

8. Why Zookeeper is Useful

It helps the system:

- Discover available servers
- Detect failed servers
- Balance user load
- Scale dynamically

9. High-Level Service Discovery Flow



10. Step-by-Step Workflow

Step 1: User Logs In

None

User A opens chat application

and attempts login.

Step 2: Load Balancer Routes Request

The login request goes through:

None

Load Balancer

which forwards request to API servers.

Step 3: User Authentication

Backend servers verify:

- Username
- Password
- Authentication token

11. Service Discovery Chooses Chat Server

After authentication:

None

Service Discovery selects best chat server

based on predefined rules.

12. Example

Suppose:

None

Chat Server 2

is selected.

Reasons may include:

- Lowest latency
- Available capacity
- Closest geographical location

13. Server Information Returned

The system sends back:

None

DNS hostname / server address

to the client.

Example:

None

chat-server-2.example.com

14. WebSocket Connection Established

Finally:

None

User A connects to Chat Server 2

using WebSocket.

15. Why Persistent Server Assignment Matters

Once connected:

- User typically remains on same chat server
- Avoids reconnect overhead
- Preserves connection state

16. Benefits of Service Discovery

Load Balancing

Users distribute evenly across servers.

Improved Scalability

New servers can be added dynamically.

Better Reliability

Failed servers can be removed automatically.

Lower Latency

Users connect to geographically closer servers.

Efficient Resource Usage

Avoids overloaded servers while other servers remain idle.

17. Dynamic Server Registration

Chat servers usually:

None

Register themselves with Zookeeper

during startup.

18. Heartbeat Mechanism

Servers periodically send:

None

Heartbeat Signals

to indicate they are alive.

19. Failure Detection

If heartbeat stops:

None

Zookeeper marks server as unavailable

and removes it from routing list.

20. Horizontal Scaling Support

When traffic increases:

- New chat servers are added
- Servers register automatically
- Service discovery starts routing users to them

This enables:

None

Horizontal Scaling

21. Real-World Importance

Without service discovery:

- Manual server assignment becomes difficult
- Load balancing becomes inefficient
- Large-scale WebSocket management becomes unreliable

22. Relationship with Stateful Chat Servers

Since chat servers maintain:

None

Persistent WebSocket Connections

service discovery becomes extremely important.

Unlike stateless HTTP servers:

- Clients cannot switch servers frequently

23. Difference from Stateless HTTP Routing

Stateless HTTP

Each request can go to different server.

Stateful WebSocket

Persistent connection must remain stable on same server.

24. Simplified Architecture Flow

None

User → Load Balancer → API Server



Service Discovery



Best Chat Server



WebSocket Connection

25. Key Challenges in Service Discovery

Server Overload

Too many users on one server.

Connection Migration

Moving active WebSocket users is difficult.

Failure Handling

Need rapid detection of dead servers.

Geographical Routing

Users should connect to nearest region.

26. Common Optimization Techniques

- Consistent hashing
- Health checks
- Heartbeat monitoring
- Dynamic scaling
- Geo-based routing

Key Takeaway

Service discovery is responsible for selecting the most suitable chat server for a client.

Apache Zookeeper is commonly used because it:

- Tracks active servers
- Detects failures
- Balances load
- Supports scalability

After authentication:

1. Service discovery selects best server
2. Client receives server address
3. WebSocket connection is established

Since chat systems use persistent WebSocket connections, intelligent server assignment is critical for scalability, reliability, and low latency.